01 / KODU ANLAMAK

Bir vardiyadan diğerine, bilgi kaybolmadan

Vardiya Defteri; GT1, GT2 ve ST için gözlemleri, açık işleri ve dikkat konularını ekip içinde taşıyan bir web uygulaması.

> **Ekran gösterir. Sunucu kuralları uygular. Veritabanı bilgiyi kalıcı tutar.**

01 · GÖRÜNEN KISIM

React arayüzü

TSX + TypeScript + CSS
Vite, tarayıcı için JS/CSS üretir

→

02 · İSTEK KAPISI

Express API

Node.js üzerinde JavaScript
Kimlik, yetki, doğrulama, iş kuralları

→

03 · KALICI HAFIZA

PostgreSQL

Prisma + pg üzerinden erişim
Kayıtlar, ilişkiler, geçmiş, fotoğraflar

Gözlem ve işlem ayrı

Genel Bakış durumu okumak içindir. **Yeni Kayıt**, **Devir Panosu** ve **Geçmiş** ilgili işlemleri kendi alanında toplar.

İşin ömrü vardiyadan uzun

Açık iş aynı kayıt olarak kalır. Vardiya bitti diye silinmez; açıkça kapatılınca geçmişte saklanır.

AI isteğe bağlı bir yan kol

Normal kayıt akışı AI olmadan çalışır. AI metni inceleyip öneri verir; ekipman durumunu veya kayıtları kendisi değiştirmez.

Başlangıç noktası: Yaklaşık 10 kişilik ekip ve belirli bir tesis. Bu ölçek, tek sunucu ve sade arayüz tercihlerini anlamlı kılıyor. Bu rehber uygulamanın içine yeni özellik eklemez.

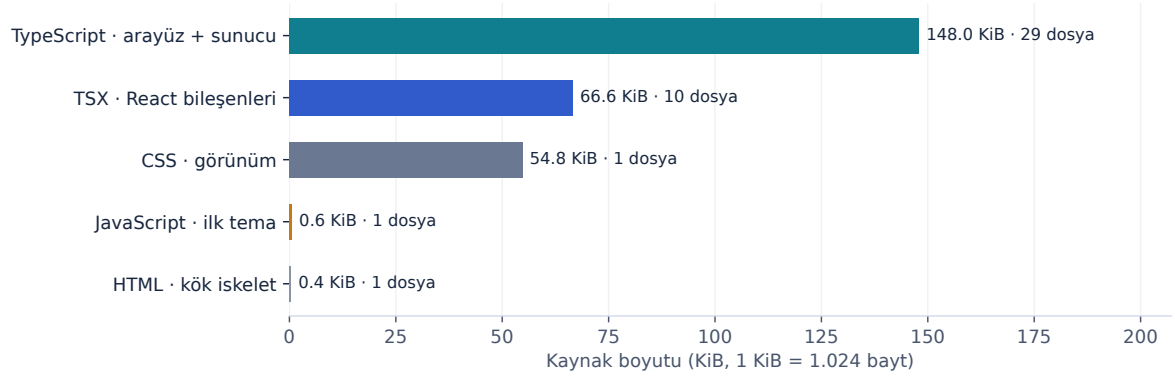
`frontend/App.tsx ↗ src/app.ts ↗ src/routes/operations.ts ↗`

02 / KODU ANLAMAK

Proje şu anda ne büyüklükte?

Bunlar kullanıcı veya satış istatistiği değil; 25 Ekim 2026 tarihinde kaynak koddan çıkarılan envanter sayıları.

4	22	13	8
ana ekran	API uç noktası	Prisma modeli	veritabanı göçü



Ölçüm: izlenen src/ ve frontend/ kaynakları; üretilmiş public/, Prisma kodu, testler, node_modules ve kilit dosyası hariç. Toplam 270.5 KiB. TypeScript hem arayüz hem sunucu kodu içerir; TSX ayrı gösterilir.

22 API = 20 korumalı + 2 giriş kapısı

`POST /login` ve `POST /register` bearer token istemez. Kayıt kapısı ayrıca ekip koduyla korunur. Statik dosyalar ve kök yönlendirme bu 22 sayısına dahil değildir.

13 model = 13 aktif özellik değil

`Handover`, `Log` ve `Report` eski şemadan korunmuş modellerdir; bugünkü aktif devir akışının merkezi `OperationEntry` modelidir.

31 doğrudan paket tanımı

15 çalışma zamanı, **16** geliştirme bağımlılığı var. React ve ReactDOM çalışırken; Vite, eklentisi ve React tipleri geliştirme/derlemede kullanılır. Bu, dolaylı paketlerin toplamı değildir.

> Kod boyutu; kalite, çalışma süresi veya harcanan emek yüzdesi değildir. HTML/CSS dosyalarında uzun satırlar bulunduğu için satır sayısı yerine bayt boyutu kullandım. GitHub dil grafiği farklı dosya kapsamı nedeniyle farklı çıkabilir.

03 / KODU ANLAMAK

Bir “Kaydet” tıklamasının yolculuğu

Örnek: “GT1 titreşim kontrolü sürüyor; mekanik ekip sonucu bekleniyor.” Aşağıdaki adımlara tıklayarak akışı izle.

React bileşeni

1. Form doldurulur

EntryForm alanı değişince AppStore taslağı güncellenir; bileşen bu durumu okuyarak görünür değeri gösterir. Yerel doğrulama sunucunun doğrulamasının yerini tutmaz.

`frontend/components/EntryForm.tsx ↗`

Tarayıcı

2. Fotoğraf hazırlanır

Desteklenen büyük görseller en uzun kenar 1.600 piksel olacak şekilde küçültülür. Orijinal seçim taslakta kalır; daha büyük bir çıktı oluşursa orijinal gönderilir.

`frontend/lib/photo-upload.ts ↗`

AppStore → HTTP

3. İstek gönderilir

saveEntry, metin ve dosyaları FormData ile POST /operations/entries adresine taşır. Authorization başlığına bellekteki bearer token eklenir. Bu örnekte AI çağrısı yoktur.

`frontend/state/store.ts ↗`

Sunucu

4. Kimin gönderdiği doğrulanır

JWT imzası ve süresi kontrol edilir. Kullanıcı ve güncel rol veritabanından okunur; tarayıcının gönderdiği authorId ya da role doğru kabul edilmez.

`src/middleware/auth.ts ↗`

Sunucu

5. İçerik denetlenir

Yazma limiti, dosya sayısı/boyutu, tür ve metin sınırları uygulanır. Sharp dosyayı gerçekten açar, döndürür ve JPEG olarak yeniden üretir; EXIF taşınmaz.

`src/routes/operations.ts ↗`

PostgreSQL

6. Tek işlem içinde yazılır

OperationRevision ilerletilir. Aktif vardiya varsa bağlantısı eklenir. Kayıt, fotoğraflar ve varsa ilk iş takibi geçmişi aynı transaction içinde kaydedilir; hata olursa birlikte geri alınırlar.

`src/routes/operations.ts ↗`

Sunucu → React

7. Başarı görünür olur

201 yanıtından sonra AppStore formu temizler ve veriyi yeniler. React güncel durumu ekrana taşır. Hata olursa metin/fotoğraf korunur; eski oturumun yanıtı yeni hesabı değiştiremez.

`frontend/state/store.ts ↗`

Ortak veri

8. Sonraki vardiya okur

Açık iş sonraki operatöre aynı kayıt olarak görünür. Açıkça kapatma işlemi closedAt, kapatan kişi ve kapanış notunu saklar. Kaydı kapatmak silmek değildir.

`src/routes/operations.ts ↗`

İki ayrım önemli: Aktif vardiya, yeni kayıt için zorunlu değildir; varsa kayıtla ilişkilendirilir. Buna karşılık “devri okudum” onayı aktif vardiya ister. Ayrıca gelişme notlarındaki tekrar koruması yeni kayıt oluşturma uç noktasında henüz yok; yanıt kaybolduktan sonraki manuel tekrar kopya kayıt üretebilir.

04 / KODU ANLAMAK

Tarayıcı tarafı artık React + TypeScript

HTML ekranları TSX bileşenlerine taşındı. CSS hâlâ yerleşim ve görünümü belirliyor; React arayüzün veri değişince nasıl güncelleneceğini düzenliyor.

React 19 + ReactDOM

React bileşenleri ekranı tanımlar; ReactDOM bu bileşenleri tarayıcıdaki root içine yerleştirir.

Neden uygun? AuthView, EntryForm, History ve ortak Field gibi parçalar birlikte tekrar kullanılabilir.

Bedeli: Ek çalışma zamanı kodu ve React kavramlarını öğrenme ihtiyacı getirir.

`frontend/main.tsx ↗` `frontend/App.tsx ↗`

TSX + TypeScript

TSX, TypeScript dosyasında ekran yapısını ifade eder. Alanlar ve API veri yapıları tiplerle açıklanır.

Neden uygun? Yanlış prop ve alan kullanımını geliştirme sırasında yakalamaya yardım eder.

Bedeli: Tipler tarayıcıda silinir; güvenilmeyen girdiyi sunucuda doğrulamak yine gerekir.

`frontend/components/EntryForm.tsx ↗`

`frontend/state/types.ts ↗`

Vite + React eklentisi

TSX/modülleri tarayıcı için JS/CSS'e dönüştürür. İsteğe bağlı geliştirme sunucusu Fast Refresh sağlar.

Neden uygun? Tek komutlu 3000 adresi korunur; ayrı geliştirme modunda hızlı geri bildirim alınır.

Bedeli: Derleme, paket güncellemeleri ve üretilen dosyaları yönetme sorumluluğu ekler.

`vite.config.ts ↗` `scripts/dev.mjs ↗`

AppStore + useSyncExternalStore

Oturum, kayıt taslağı ve istek durumu bellekteki depoda; bileşenler seçtikleri durum dilimine abone olur.

Neden uygun? Sayfa/tema/dil değişse de aynı taslak korunur. Redux eklemeden ortak durum yönetilir.

Bedeli: Store düzeni, sabit snapshot referansları ve asenkron yarışlar uygulamanın sorumluluğudur.

`frontend/state/store.ts ↗` `frontend/state/hooks.ts ↗`

CSS, i18n ve Intl

CSS temayı/kontrastı; i18n.ts TR/EN metinlerini yönetir. Bağlantı ve tesis saatleri Europe/Istanbul kullanır.

Neden uygun? Aynı bileşen iki tema ve dilde çalışır. Yalnız tercihler localStorage'da saklanır.

Bedeli: Yeni renk ve metinler her tema, dil ve mobil genişlikte kontrol edilmelidir.

`frontend/styles.css ↗` `frontend/lib/i18n.ts ↗`

Fetch, FormData ve iptal koruması

HTTP ve fotoğraf taşıma yine tarayıcı API'leriyle yapılır. AbortController ve istek/oturum sürümleri eski yanıtları eler.

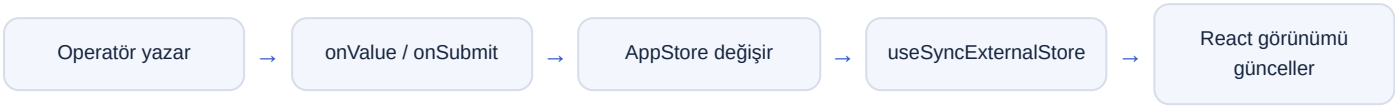
Neden uygun? Mevcut endpoint ve veri sözleşmeleri değişmeden kullanılır; token/taslak diske yazılmaz.

Bedeli: React tek başına ağ güvenliği, çevrimdışı kayıt veya tekrar güvenliği sağlamaz.

`frontend/state/store.ts ↗`

React neyi kolaylaştırıyor? Tek yönlü veri akışı

Bir alanı elle DOM'dan arayıp değiştirmek yerine, kullanıcı eylemi durum günceller; bileşen yeni duruma göre ekranı tarif eder.



Gerçek koddaki yaklaşım

```
const entry = useApp(s => s.entry);

<Field
  value={entry.title}
  onValue={title =>
    app.changeEntry({ title })}
/>
```

EntryForm'dan sadeleştirilmiş örnek. Tek kaynak AppStore'daki taslaktır; veri akışı bu yüzden izlenebilir.

`frontend/components/EntryForm.tsx ↗`

Kullanıcı açısından karşılığı

Tema veya dil değişince alanın DOM kimliği ve imleç seçimi korunur. Sayfalar arasında dolaşınca metin ve orijinal fotoğraf seçimi yerinde kalır.

Ortak Field bileşeninde yapılan erişilebilirlik/hata mesajı düzenlemesi birçok formda aynı davranışı sağlar.

`tests/e2e/react-frontend.spec.ts ↗`

useRef / useEffect

useRef: alanı odaklamak veya dialog gibi tarayıcı nesnelerine erişmek. **useEffect:** dinleyici/poll kurmak ve ayrılırken temizlemek. Her şey için useEffect kullanmak gerekmez.

Key ve memo ne yapar?

Kararlı key, React'in listedeki kaydı tanımasını sağlar. memo ve değişmeyen veri referansları bazı gereksiz render'ları azaltabilir. Bunlar otomatik hız garantisi değildir.

StrictMode ne yapmaz?

Geliştirmede yaşam döngüsü hatalarını görünür kılar; üretimde güvenlik katmanı değildir. Effect temizliği ve eski istek koruması yine kodda kurulmalıdır.

Oturum ve taslak sınırı

Tercihler kalıcıdır; token ve özel taslak bellektedir. Yenileme oturumu bitirir. Süre dolunca taslak yalnız aynı hesaba geri verilir; açık çıkışta silinir.

Kaynak dosyası ile tarayıcı çıktısını ayırmak

Geliştirirken frontend/ düzenlenir. Kullanıcı TSX indirmez; Vite'ın ürettiği JavaScript ve CSS'i alır. Express bu çıktıları aynı /app/ adresinden sunar.



Alışılmış tek komut

```
npm run dev
# localhost:3000/app/
```

Vite önce ilk çıktıyı hazırlar, sonra API başlar. Kaynaklar izlenir; bu moddaki tarayıcı sayfasını değişiklikten sonra yenilersin.

Yayın için

```
npm run build
npm start
```

Fast Refresh · iki terminal

```
npm run dev:api
# Ayrı terminalde:
npm run dev:web
# localhost:5173/app/
```

Vite API isteklerini 3000'e yönlendirir. Uygun bileşen değişiklikleri form durumunu koruyabilir; her değişiklikte koruma garantisi yoktur.

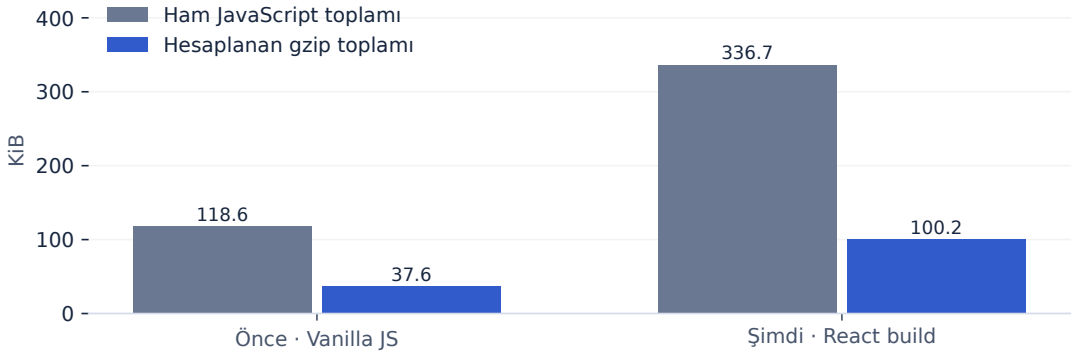
> **Gizli ayarlar sunucuda kalır.** Vite sunucunun kök .env dosyasını okumaz. JWT_SECRET, REGISTRATION_CODE ve OPENAI_API_KEY tarayıcıya gömülmez. Kullanıcının girdiği kod kayıt isteğinde gönderilir; canlıda HTTPS ile korunmalıdır.

public/ Git'te tutulmaz; npm test ve test:integration kendi statik varlıklarını önceden üretir. Entegrasyon CI'sindeki eksik hazırlık adımı, güvenlik testini gevşetmeden bu şekilde düzeltildi.

vite.config.ts ↗ package.json ↗

Önce / sonra: kazanım ve maliyet birlikte

React geçişinin temel kazanımı bakım düzeni. Daha küçük dosya, daha yüksek hız veya daha güvenli API kendiliğinden gelmez.



Eski kod: 460817a public/ içindeki 11 JS dosyası. Yeni build: 5bd019c içindeki 3 üretilmiş JS dosyası. Gzip boyutu dosya başına hesaplandı; sunucuda sıkıştırma etkin olduğu veya sayfanın bu kadar hızlı yüklendiği iddiası değildir.

Konu	Önce	Şimdi
Ekranı güncelleme	DOM seçimi + olay dinleyicileri + elle metin güncelleme	TSX bileşenleri + durum abonelikleri
Form düzeni	Formlar/modüller boyunca elle uyum	Ortak Field ve tipli veri yapıları
Geliştirme	Doğrudan statik dosya düzenleme	Vite build/watch; isteğe bağlı Fast Refresh
API ve veri	Express + Prisma + PostgreSQL	Aynı endpoint ve şema; yeni migration yok
Kullanıcı özellikleri	Fotoğraf, Excel, AI, TR/EN ve tema	Korundu; React bu özellikleri ilk kez getirmiş sayılmaz

Bir sonraki performans kararı ölçümle verilir. İlk yükleme/bundle boyutu arttı. Gerçek telefonda açılış süresi, etkileşim gecikmesi ve yavaş ağ ölçülmeden “daha hızlı” dememek gerekir. Gerekirse AI gibi seyrek bölümler için parça yükleme ve proxy gzip/Brotli ayrıca değerlendirilebilir.

`docs/reviews/2026-10-25-react-migration.md ↗`

Sunucu ve veri: teknolojiler birbirinin yerine geçmez

Node.js çalıştırır, Express yönlendirir, TypeScript hata yakalamaya yardım eder, Prisma sorgu kurar, PostgreSQL kalıcı ve tutarlı veri saklar.

Node.js

Sunucudaki JavaScript'i çalıştıran ortamdır. Geliştirmede tsx watch; yayında derlenmiş dist/index.js kullanılır.

Neden uygun? Tarayıcıyla aynı dil ailesi; ağ ve veritabanı işleri için async/await akışı uygun.

Bedeli: CPU yoğun görsel işleri ve eşzamanlı yük kaynak bütçesi ister.

`package.json ↗`

TypeScript

Fonksiyon ve veri tiplerini derleme sırasında denetler. tsc sunucu kaynaklarını JavaScript'e çevirir.

Neden uygun? strict kontroller, yanlış alan/tip kullanımını daha erken gösterir.

Bedeli: HTTP'den gelen veriyi çalışma anında otomatik doğrulamaz.

`tsconfig.json ↗`

Express 5

URL + HTTP yöntemi ile ilgili handler'ı bulur; middleware zinciri kimlik ve güvenlik kontrollerini ekler.

Neden uygun? Küçük servis için açık, esnek bir akış sunar.

Bedeli: Katman düzeni ve doğrulama disiplini geliştiriciye kalır.

`src/app.ts ↗`

Prisma + pg adapter

Şemadan tipli client üretir. PrismaPg/pg, PostgreSQL bağlantısını sağlar; migration dosyaları şema değişikliklerini taşır.

Neden uygun? İlişkili kayıtlar ve tipli sorgular daha okunabilir olur.

Bedeli: Üretilen SQL ve ORM davranışı da öğrenilmeli; SQL bilgisi hâlâ gerekli.

`src/database.ts ↗` `prisma/schema.prisma ↗`

PostgreSQL

Kullanıcı, vardiya, iş, geçmiş ve fotoğraf baytlarını ilişkili tablolarda kalıcı saklar.

Neden uygun? Transaction, yabancı anahtar, benzersizlik ve satır kilidi tutarlılığı destekler.

Bedeli: Sunucu, yedekleme ve migration işletimi gerektirir.

`prisma/schema.prisma ↗`

Multer + Sharp

Multer multipart yüklemeyi alır. Sharp gerçek görseli doğrular, döndürür, küçültür ve yeniden kodlar.

Neden uygun? İstemcinin dosya türü iddiasına güvenmek yerine sunucuda yeniden kontrol eder.

Bedeli: Bellekte yükleme ve görsel işleme maliyeti vardır.

`src/routes/operations.ts ↗`

ExcelJS

Yetkili vardiya sorgusunu gerçek XLSX dosyasına çevirir. Metin hücreleri formül olarak yorumlatılmaz.

Neden uygun? Operatörün alışık olduğu dosya formatını verir.

Bedeli: Büyük export bellek/CPU kullanır; bu yüzden 5.000 satır sınırı var.

`src/services/export.service.ts ↗`

dotenv + config fonksiyonları

Ayarları sunucunun ortamından okur; eksik/zayıf uzunlukta secret ayarlarında başlangıcı durdurur.

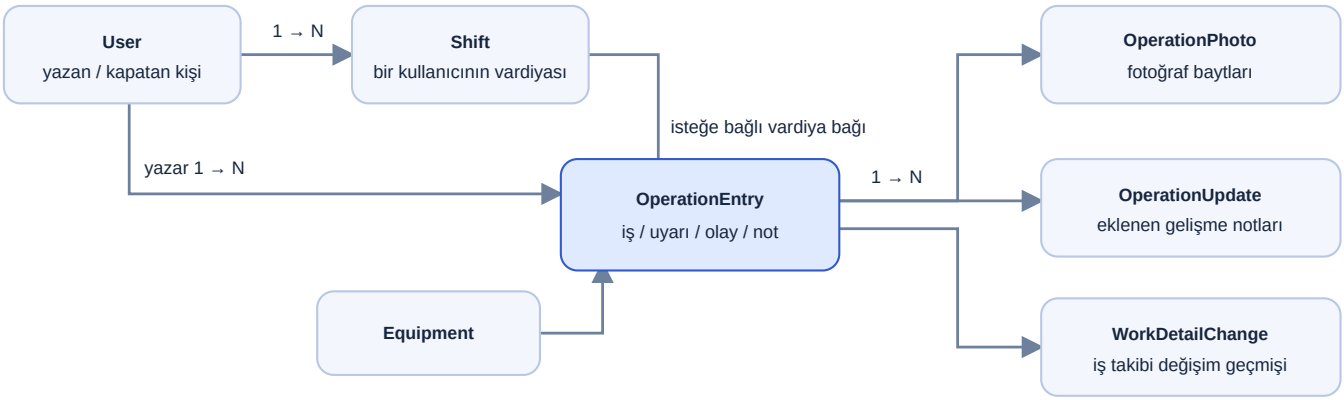
Neden uygun? Gizli değerler kaynak koddan ayrılır; testler bağımlılıkları enjekte edebilir.

Bedeli: dotenv şifreleme sağlamaz; .env dosyası ayrıca korunmalı.

`src/config.ts ↗`

Veri modeli: merkezde vardiya değil, kayıt var

İşler vardiyalar arasında yaşayabildiği için kayıt ile vardiya aynı şey olarak tasarlanmadı. Bu şekil önemli bağları gösterir; şemanın tamamı değildir.



OperationEntry

Başlık, açıklama, tür, öncelik ve kapanış bilgisini taşır. `shiftId` ve ünite bağı isteğe bağlıdır. TASK/VALVE/ATTENTION açık kalır; EVENT/NOTE geçmiş görünümüne gider.

Geçmiş ayrı kayıtlar halinde

`OperationUpdate` eklenen notu; `WorkDetailChange` iş durumu/ekip/sonraki adım değişimini saklar. Eski metin ve ilk yazan kişi kaybolmaz.

HandoverRead

“Okudum” onayını kullanıcı + aktif vardiya + veri sürümüyle bağlar; özetin o andaki kopyasını saklar. Yeni bilgi gelirse önceki onay yeni durumu kapsamış sayılmaz.

OperationRevision

Operasyon yazmalarında artan ortak sürümdür. Onay ve eşzamanlı değişikliklerin aynı tutarlı veri üzerinde değerlendirilmesine yardım eder.

> **Neden ilişkisel veritabanı?** “Bir fotoğraf hangi kaydın?”, “Bir onay hangi vardiyanın?” soruları açık bağlar ister. Yabancı anahtarlar yetim veriyi; transaction’lar yarım kaydı önlemeye yardım eder. SQL tabloları bir Excel dosyası gibi elle senkron tutulmaz.

`prisma/schema.prisma ↗` `src/services/handover-summary.ts ↗`

İki kişi aynı kaydı değiştirirse ne olur?

Ekranda butonu devre dışı bırakmak aynı tarayıcıdaki çift tıklamayı azaltır. İki farklı bilgisayar için sunucu ve veritabanı koruması gerekir.

ÖĞRETİCİ SİMÜLASYON · VERİTABANINA BAĞLANMAZ

Ayşe ve Can aynı işin **sürüm 4** halini açtı. Farklı bilgiler kaydetmek istiyorlar.

Henüz değişiklik yok. Sunucudaki sürüm: 4.

İyimser sürüm kontrolü

Güncelleme beklenen sürümü taşır. Başkası farklı içerikle önce kaydettiyse eski istek **409** alır; taslak korunur. Son gelenin sessizce kazanması veri kaybettirebilirdi.

`src/services/work-details.ts ↗`

Transaction + satır kilidi

Değişiklik, geçmiş ve ortak sürüm birlikte işlenir. Ortak revision satırı üzerinde kilit, eşzamanlı kapatma/onay ile çelişen yazmaları sıralar.

`src/services/operation-updates.ts ↗`

Tek aktif vardiya: DB kuralı

Koşullu unique index aynı kullanıcı için yalnızca ACTIVE satırlarında benzersizlik ister. Tamamlanmış vardiyalar bu kuralla engellenmez.

`prisma/migrations/20261019000100_one_active_shift_per_user/migration.sql ↗`

Tekrar güvenliği sınırlı kapsamda

Gelişme notlarında `authorId + requestId` aynı gönderimin iki kez yazılmasını önler. Bu koruma her yazma uç noktasında varmış gibi düşünülmemeli.

`src/services/operation-updates.ts ↗`

12 eşzamanlı başlatma → 1 başarı + 11 çatışma

İki bağımsız API/Prisma örneğiyle test edilen beklenti; 25 Ekim entegrasyon turunda geçti. Bu sayı bir hız veya kapasite ölçümü değildir.

Bedel: Ortak revision kilidi basit ve tutarlı; yazma sayısı çok büyürse darboğaz olabilir. Daha büyük kurulumda kilit kapsamını daraltmak gerekir; küçük ekip için ilk tercih mikroservis değildir.

Güvenlik tek bir paket değil, bir kontrol zinciri

Girişte kullanılan parola, hesap açma kodu ve oturum token'ı farklı şeylerdir. Aynı amaçla saklanmaz ve karşılaştırılmazlar.

Parola → scrypt + rastgele salt

Parola düz metin tutulmaz. Her hesap için rastgele salt ve maliyetli bir türetim kullanılır. Şifreyi geri açmak yerine aday parolanın sonucu karşılaştırılır.

`src/password.ts ↗`

Kayıt kodu → kapalı varsayılan

Kod yapılandırılmamışsa kayıt 403 ile kapalıdır. Tanımlı kod 12–128 karakterdir. SHA-256 özetleri timingSafeEqual ile karşılaştırılır; yeni rol zorla WORKER olur.

`src/app.ts ↗`

JWT → imzalı oturum belgesi

HS256 imzası ve süre kontrol edilir. JWT şifreleme değildir; payload gizli bilgi taşımamalı. Tarayıcı token'ı bellekte tutar; sunucu güncel kullanıcı/rolü ayrıca DB'den okur.

`src/middleware/auth.ts ↗`

Yetki → işe göre sınır

Operasyon kayıtları aynı ekipte ortaktır. Kişisel vardiya sorgusu kendi kullanıcı kimliğini kullanır; tüm vardiyaları/kişileri listelemek MANAGER ister. Bu sistem çok kiracılı bir SaaS değildir.

`src/services/shift-query.ts ↗`

Helmet + güvenli sunum

CSP ve başlıklar tarayıcı korumalarını destekler. React'te kayıt metni normal metin düğümü olarak gösterilir; ham HTML'e çevrilmez. Fotoğraflar yetkili route'tan gelir; hata yanıtları iç ayrıntıları vermez.

`src/app.ts ↗`

Deneme ve boyut limitleri

15 dakikada IP başına giriş 20, kayıt 10, operasyon yazması 60; JSON gövdesi 16 KB. AI için kullanıcı başına ayrı limit ve eşzamanlı çağrı sınırı bulunur.

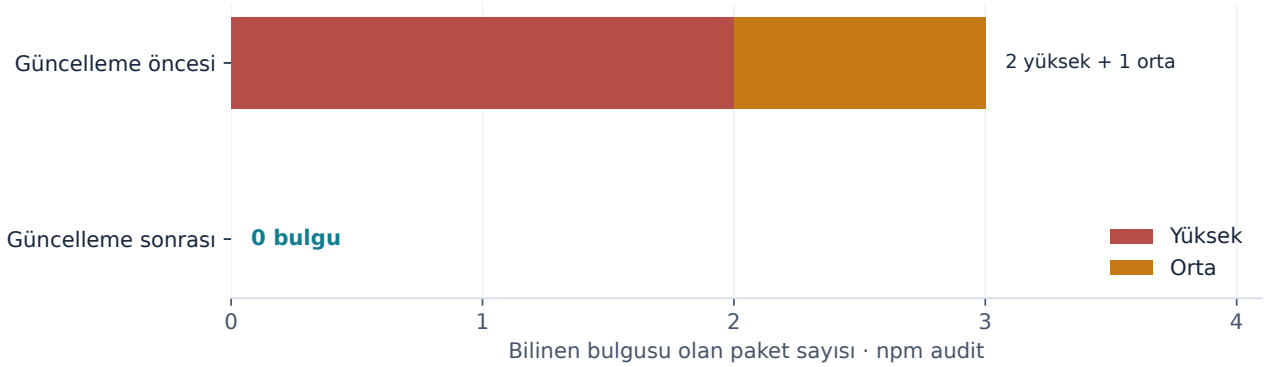
`src/routes/operations.ts ↗`

> **Bilinen sınır:** Çıkış tarayıcı belleğini temizler ama verilmiş JWT'yi sunucuda iptal etmez. Önceden ele geçirilmiş token 8 saatlik ömrünün kalanında çalışabilir. Canlı kullanım öncesi kalıcı oturum iptali önceliklidir. HTTPS gerekir; CSP tek başına ağ trafiğini şifrelemez.

`docs/reviews/2026-10-24-security-assessment.md ↗`

Son güvenlik düzeltmesi: uyarıdan doğrulanmış yamaya

25 Ekim’de iki yüksek ve bir orta seviyeli paket uyarısı vardı. Uyumlu minor/patch sürümlerle güncellendi; uygulama kuralları ve CI eşikleri değiştirilmedi.



Kaynak: aynı npm audit komutunun güncelleme öncesi ve sonrası JSON çıktısı. Üç etkilenen paketin sayısıdır; üç saldırı veya tüm uygulamanın güvenlik puanı değildir.

Paket	Önce → sonra	Görevi / giderilen sorun
body-parser	2.3.0 → 2.4.0	Express’in JSON okuyucusu; sıkıştırılmış istek sonundaki bozuk veriyle süreç çökmesi.
multer	2.4.0 → 2.5.0	Dosya yükleme; bekleyen asenkron filtre sırasında bağlantı kapanmasıyla DoS.
fast-uri	3.1.8 → 3.1.9	Prisma araç zincirindeki URI ayrıştırma/normalleştirme sorunları.
qs	6.16.0 → 6.16.1	Yeni body-parser sürümünün gerektirdiği dolaylı yama.

Tarama sonucundan daha güçlü kanıt

İzole çocuk süreçte eski body-parser ile Gunzip Z_DATA_ERROR çökmesi tekrarlandı. Aynı test yeni sürümde geçti; sonraki istek işlendi ve açılmış gövdenin 16 KB sınırı korundu.

Neden lockfile önemli?

package-lock.json tam sürüm ağacını kaydeder. npm ci bu yamaları tekrarlanabilir biçimde kurar. Denetimi susturmak yerine paketler düzeltildi; yeni major veya override eklenmedi.

0 audit bulgusu, kusursuz uygulama demek değildir. Sonuç bilinen paket duyurularını kapsar. Yetki, yanlış iş kuralı, oturum iptali ve canlı ortam güvenliği ayrıca değerlendirilir. Mevcut filtre senkron olsa da etkilenen Multer sürümü kaldırıldı.

docs/reviews/2026-10-25-dependency-security.md ↗ tests/dependency-security.test.ts ↗

AI nerede devreye giriyor; nerede duruyor?

Burada kendi modelimizi eğitmedik. OpenAI SDK üzerinden dış bir modele istek atan, cevabı doğrulayan ve operatöre öneri sunan bir modül var.

Metin + dil → Sunucu doğrulaması → OpenAI Responses API → Şema + kanıt kontrolü → Operatör seçer ve kaydeder

Giden veri

Sohbette biriken kayıt metni, varsa içe alınan başlık ve dil gönderilir. Fotoğraf, parola, token ve tüm tesis geçmişi bu analize eklenmez. Operatörün serbest metne ne yazdığı yine önemlidir.

Dönen veri

Ünite, olay kategorisi, kayıt türü, **insan inceleme önceliği**, gerekçe, belirsizlik, alıntılar ve eksik bilgi soruları gelir. Katı JSON Schema'ya ek olarak sunucu tekrar kontrol yapar.

Kanıt ≠ doğruluk garantisi

Alıntıların verilen metinde geçmesi zorunludur. Bu, uydurma alıntıyı azaltır; sınıflandırmanın doğru olduğunu kanıtlamaz. Saha kayıtları ve uzman etiketleriyle doğruluk henüz ölçülmedi.

Operatör son kararı verir

Öneriyi forma uygulamak kayıt oluşturmaz. Ayrı Kaydet işlemi gerekir. Modelin ekipman kontrol aracı veya otomatik vana/duruş yetkisi yoktur.

10
kullanıcı başına AI isteği / 15 dk

3
uygulama örneği başına eşzamanlı AI çağrısı

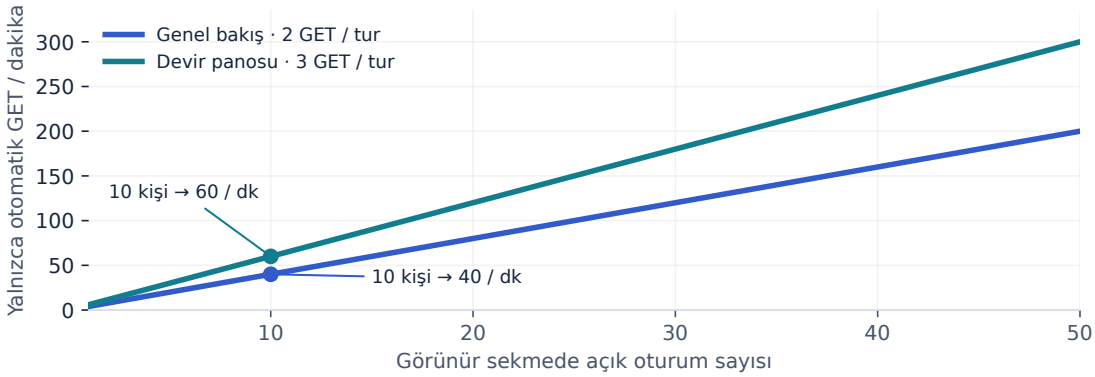
22 sn
route iptal zamanlayıcısı; SDK 20 sn

Neden bağımsız ve isteğe bağlı? API bakiyesi, ağ veya sağlayıcı sorunu normal kayıt işini durdurmamalı. ChatGPT aboneliği API bakiyesi sağlamaz. Model varsayılanı gpt-4.1-mini; ortam ayarıyla değişebilir. store:false istenir ama bu, sağlayıcının hiç veri tutmadığının garantisi değildir.

`src/routes/ai.ts ↗` `src/services/ai-analysis.ts ↗`

Performans: tahmin edebildiğimiz ile ölçmediğimiz

Veri yalnız ilgili sayfada alınır. Sayfalama, kararlı veri referansları ve seçili React abonelikleri gereksiz işi azaltmaya yardım eder; React'e geçiş tek başına hız garantisi değildir.



Hesap modeli: görünür ve giriş yapılmış sekmeler, mevcut 30 saniye aralığı, Genel Bakış için tur başına 2 GET; Devir Panosu için 3 GET. Sunucu hızı veya veritabanı sorgu sayısı grafiği değildir.

İstek sayısıyla oyna

40 GET / dakika

10 görünür oturum · Genel Bakış · 30 sn · mevcut aralık

Formül: oturum × (60 / saniye) × tur başına GET. 15/60 saniye seçenekleri yalnız senaryodur; uygulamanın ayarını değiştirmez.

Daha az gereksiz veri

Arka plan yenilemesi yalnız görünür Genel Bakış/Devir sayfalarında çalışır. Geçmiş ve Yeni Kayıt düzenli poll yapmaz. Liste sayfası 20 kayıt; fotoğraf baytları liste yanıtında yok.

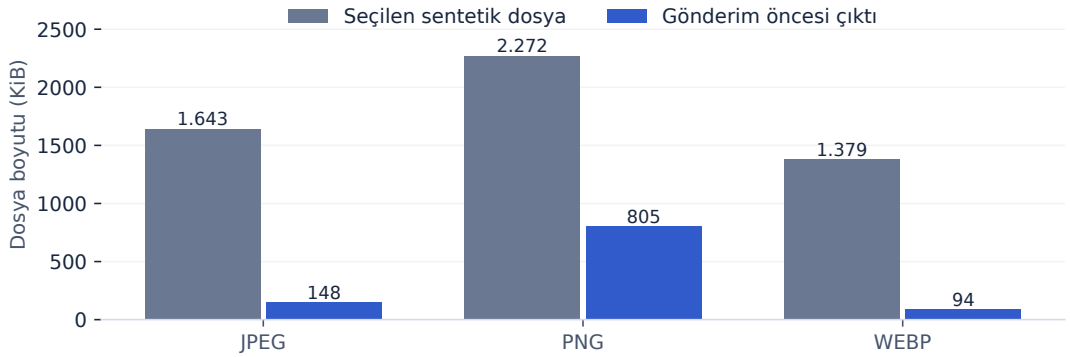
Henüz ölçülmeyenler

Gerçek ekip kullanımında p95 yanıt süresi, eşzamanlı kapasite, CPU/RAM ve saha ağ hızı ölçülmedi. Grafik bunların yerine geçmez; ilk yükleme, manuel yenileme ve yazma istekleri hesaba katılmaz.

`frontend/state/store.ts ↗` `frontend/state/hooks.ts ↗`

Fotoğraf küçültme: bu kez gerçek bir deney

25 Ekim'de uygulamanın gerçek preparePhoto fonksiyonunu Chromium'da çalıştırdım. Aynı üretilmiş 2.400 × 1.800 piksel desen JPEG, PNG ve WebP olarak denendi.



Gösterilen çıktı yalnız tarayıcının gönderim öncesi dosyasıdır. Sunucu sonradan ayrıca JPEG üretir; depolanan fotoğraf boyutu bu grafikle aynı olmak zorunda değildir.

Biçim	Giriş · KiB	Çıktı · KiB	Bu örnekte azalma
JPEG	1 643.4	148.3	%91
PNG	2 272.1	805.0	%64,6
WEBP	1 379.1	93.6	%93,2

Piksel sayısı

4.320.000 → 1.920.000 piksel: **%55,6 daha az piksel**. Üç çıktı da 1.600 × 1.200 oldu. Dosya boyutu değişimi ise biçim ve sıkıştırma ayarına da bağlıdır.

İstemci ve sunucu neden birlikte?

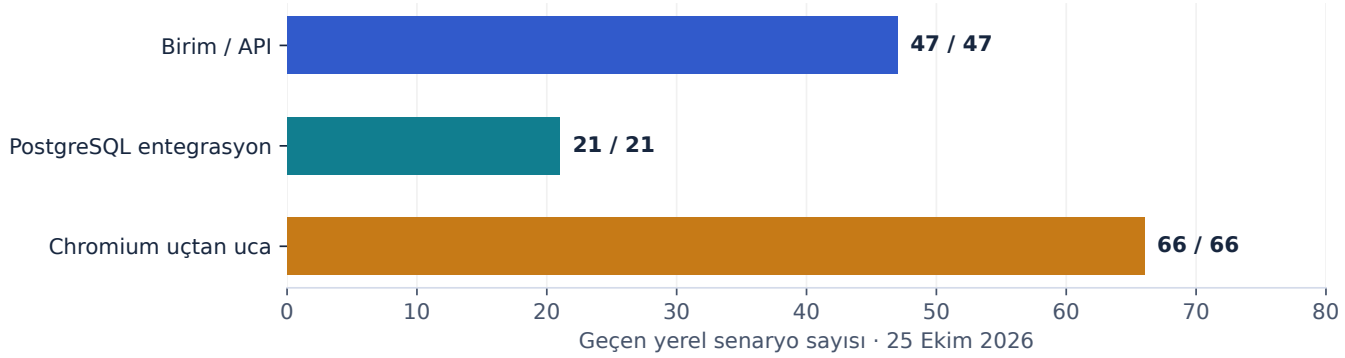
Tarayıcı küçültmesi yüklenen baytı azaltır. Tarayıcı denetimleri atlanabildiği için Multer/Sharp ile sunucuda doğrulama ve yeniden üretim yine gerekir.

Bu yüzdeler saha ortalaması değil. Tek deterministik desen kullanıldı; JPEG/WebP giriş kalitesi 95, uygulamanın canvas kalitesi 0,8; PNG kayıpsız. Gerçek telefon fotoğraflarında azalma değişir. Süre, görsel kalite puanı veya ağ hızlanması ölçülmedi.

`frontend/lib/photo-upload.ts ↗` `src/routes/operations.ts ↗`

Testler ve CI: bir değişikliği nasıl güvenle taşıyoruz?

Testler tek bir “çalışıyor” kontrolü değildir. Kuralları, gerçek veritabanı davranışını ve kullanıcının tarayıcıda yaptığı işlemleri farklı katmanlarda sınarlar.



25 Ekim güvenlik güncellemesi: 47 + 21 + 66 = 134 geçen senaryo. Entegrasyon, eksik public/ ile başlatılan temiz kurulumdan sonra tekrar 21/21 geçti. Kapsam yüzdesi veya hatasızlık garantisi değildir.

checks

npm ci → Prisma → tip kontrolü → varlık üret → 47 test → audit

Node runner, tsx, Supertest; gzip/deflate regresyonu eklendi

integration

PostgreSQL 17 → migration → varlık üret → 21 test

Gerçek constraint, transaction ve statik güvenlik başlıkları

e2e

Geçici DB → build → Chromium → 66 senaryo

14 dosya, dosya başına yeni uygulama sunucusu

Neden her tarayıcı dosyasında yeni sunucu?

Testler tek sunucudaki üretim giriş limitini topluca tüketmişti. Çözüm güvenlik limitini gevşetmek yerine test sunucularını izole etmek oldu.

CI’de bulunan hazırlık eksikliği

İlk React CI turunda entegrasyon işi 20/21 geçti; /app/ için üretilmemiş dosyalar 404 verdi. pretest:integration artık önce arayüzü derler. Güvenlik testi korunarak temiz kurulum doğrulandı.

GitHub CI durumu: 5bd019c için checks, integration ve e2e başarılı (Actions #19, 25 Ekim). Yerel test sayıları ile uzak iş sonucu ayrı kanıtlardır. Dependabot haftalık npm/Actions PR’ları açar; CI kontrol eder, otomatik yayın veya birleşme yapmaz.

`.github/workflows/ci.yml ↗ .github/dependabot.yml ↗ docs/reviews/2026-10-25-dependency-security.md ↗`

Alternatifler: biri her koşulda “en iyi” değil

Aşağıdaki değerlendirme 10 kişilik tesis ekibi hedefi içindir. Puanlar uydurmak yerine, her tercihin getirdiği faydayı ve bedeli gösteriyorum.

Konu / bugün	Bugünkü tercihin artı ve eksisi	Alternatif ne zaman anlamlı?
Tek uygulama Express API + derlenmiş React	Aynı origin, tek dağıtım ve az işletim yükü. Frontend’in build adımı artık gerekir.	Mikroservis: bağımsız ölçek/dağıtım sağlar; ağ hataları ve işletim yükü ekler. Bağımsız büyüyen servisler varsa düşün.
Arayüz React + TSX + Vite	Ortak form/dialog parçaları, tipli props ve belirgin veri akışı. Bundle, bağımlılık ve öğrenme maliyeti arttı.	Vanilla JS: çok küçük ekranda az araç; karmaşık DOM/durum elle yönetilir. Vue/Svelte de bileşen sunar. React tercihi en hızlı olduğu ölçüldüğü için yapılmadı.
Veritabanı PostgreSQL	İlişkiler, transaction ve eşzamanlılık kuralları güçlü; kurulum/yedekleme gerekir.	SQLite: yerel demo için kolay; çok yazarlı sunucu büyümesinde yazma düzenini iyi değerlendirmek gerekir. MongoDB: belge esnekliği iyi; bu ilişkili modelde bağları ayrıca yönetmek gerekir.
Veri erişimi Prisma + gerektiğinde tipli SQL	Şema/migration ve tipli sorgu verimli; ORM’nin sorgu davranışını da takip etmek gerekir.	Doğrudan SQL/pg: tam kontrol verir; eşleme ve migration düzeni daha çok elde yazılır. Sorgu gereksinimleri karmaşıklaştığında seçili alanlarda düşün.
Yenileme 30 saniyelik görünür sayfa poll’u	Kurulumu basit; anlık değil ve değişiklik olmasa da istek üretir.	SSE: sunucudan tek yönlü olay iletimine uygun. WebSocket: çift yönlü akış için güçlü. İkisinde de yeniden bağlantı/ölçekleme gerekir; saniyelik gecikme yetmezse düşün.

> Daha yeni veya daha büyük teknoloji kullanmak tek başına daha iyi mühendislik değildir. Gerekçeli, ölçülmüş ve açıklanabilir seçim; başvuruda teknoloji sayısından daha güçlü bir savunmadır.

Alternatifler: güvenlik, görsel ve AI kararları

Bazı seçimler yalnız kod miktarını değil, güvenlik sorumluluğunu ve işletim maliyetini de değiştirir.

JWT / sunucu oturumu

Bugün: bellekte JWT ve her istekte DB kullanıcı kontrolü. Kurulum sade, fakat logout iptali yok.

Alternatif: DB/Redis tabanlı oturum, iptal ve oturum listesini kolaylaştırır; ilave veri yönetimi ister. HttpOnly cookie JS ile token okumayı azaltır; SameSite/CSRF politikası doğru kurulmalıdır. Mevcut JWT + DB rol sorgusu, her isteğin tamamen DB'siz olduğu anlamına gelmez.

scrypt / Argon2id / bcrypt

Bugün: Node crypto içindeki scrypt, salt ve sabit zamanlı kıyas; ayrı parola paketi gerekmez.

Alternatif: Argon2id modern bir seçenektir; paket/dağıtım ve kaynak ayarı gerektirir. bcrypt yaygındır ama 72 bayt sınırı gibi davranışları bilinmelidir. Mevcut maliyet ayarları için üretim kaynaklarında ölçüm yapılmadan "en hızlı/en güvenli" iddiası kurulamaz.

Fotoğraf DB'de / nesne deposunda

Bugün: PostgreSQL bayt alanı, özel yetkili route ve tek yedekleme bütünlüğü. Küçük kurulum sade; DB boyutu ve yedek süresi büyüyebilir.

Alternatif: S3 benzeri depo büyük fotoğraf hacmini ayırır; dosya yetkisi, silme tutarlılığı, URL ve ayrı yedekleme politikası ister. Fotoğraf hacmi gerçek darboğaz olduğunda düşün.

Bulut AI / yerel model / kurallar

Bugün: yönetilen API, eğitimsiz başlangıç ve kullanım başına ücret; dış ağa ve sağlayıcıya bağımlı.

Alternatif: yerel model dış aktarımı azaltır; donanım ve bakım ister, otomatik olarak daha doğru değildir. Anahtar kelime kuralları ucuz/açıklanabilir; bağlamı kaçırabilir. Aynı uzman etiketli veriyle hata türlerini karşılaştırmadan kazanan ilan edilmemeli.

Elde doğrulama / şema kütüphanesi

Bugün: açık if/enum/uzunluk kontrolleri; az bağımlılık. Kontroller farklı yerlerde unutulabilir. NUL hatası bunun somut örneğiydi.

Alternatif: Zod/Joi benzeri ortak şema, tekrarları azaltır ve tiplerle yakın çalışabilir; ek kütüphane ve geçiş işi getirir. Endpoint sayısı arttıkça daha değerli olur.

Dosya sınırı / bütün uygulama bütçesi

Bugün: 3 × 5 MB dosya sınırı, 20 milyon piksel sınırı ve yazma limiti. Bunlar aynı anda çok yüklemenin toplam belleğini tek başına sınırlamaz.

Alternatif: multipart öncesi sınırlı eşzamanlı kabul ve proxy limitleri. Daha geniş erişim öncesi uygulanabilir; burada kapasite testi yapıldı iddiası yoktur.

Kodu okumak için en mantıklı sıra

Her dosyayı baştan sona ezberlemek yerine tek davranışı takip et. “Buton neyi çağırıyor, sunucu neyi doğruluyor, DB neyi garanti ediyor?” sorularını birlikte sor.

1. **frontend/main.tsx + App.tsx** — React’in başlaması, dört ekran ve ortak görünüm.
2. **components/EntryForm.tsx + common.tsx** — değer/eylem bağlantısı ve ortak form parçaları.
3. **state/store.ts + hooks.ts** — taslak → FormData → API → snapshot → React aboneliği.
4. **src/index.ts + app.ts + middleware/auth.ts** — yapılandırma, kayıt kodu, token ve güncel DB rolü.
5. **routes/operations.ts + services/** — dosya, kayıt, kapatma, geçmiş ve transaction.
6. **prisma/schema.prisma + migrations/** — kodun altındaki veri kuralları.
7. **tests/ + vite.config.ts + package.json** — beklenen davranışlar ve kaynakların nasıl çalıştırıldığı.

Bağımlılıkları dışarıdan vermek

```
createApp(prisma, secret, {
  registrationCode,
  analyzer
})
```

Gerçek DB yerine kontrollü test bağımlılığı verilebilir. Kayıt kodunu createApp içinde ortamdan gizlice okumamak, testin hangi ayarla çalıştığını açık tutar.

src/app.ts ↗

Verinin güvenilir kaynağı

```
// Kimliği istek gövdesinden alma.
authorId: req.user.id

// Kullanıcıyı doğrulayıp
// güncel rolü DB’den oku.
```

Bu, mevcut koddaki yaklaşımın sadeleştirilmiş gösterimidir; kopyalanacak tam fonksiyon değildir. Tarayıcı devre dışı bırakılarak da istek gönderilebilir.

src/middleware/auth.ts ↗

> **Somut bir geliştirme kararı:** NUL içeren 8 girdi önce PostgreSQL’e ulaşıp 500 üretti. Önce bunu yakalayan test yazıldı; sonra mevcut doğrulayıcılara dar kapsamlı kontrol eklenip 400’e çevrildi. Büyük refactor yerine, hatayı kanıtlayan küçük düzeltme tercih edildi.

tests/integration/security-assessment.test.ts ↗

Projeyi nasıl anlatır, nasıl geliştirirdim?

Öncelik, yeni teknoloji eklemekten önce mevcut sorunun çözümünü güvenilir ve gösterilebilir hale getirmek.

01 · ÖNCE GÜVEN

Oturum iptali

Logout sonrası kopyalanmış token'ın da çalışmamasını sağlayan kalıcı oturum düzeni ve tekrar kullanım testleri.

02 · SONRA TEKRAR GÜVENLİĞİ

Yeni kayda requestId

Yanıtı kaybolmuş bir kayıt isteği tekrar edildiğinde kopya oluşturmamak. Gelişme notlarındaki yaklaşımı kapsamına uygun genişletmek.

03 · ÖLÇ, SONRA SEÇ

Saha ve AI değerlendirmesi

İzinli anonim kayıtlar, uzman değerlendirmesi, kritik olay kaçırma oranı ve operatörün inceleme süresini ölçmek. Önce sentetik örneklerin sınırını kabul etmek.

İş görüşmesinde 30 saniye

"Endüstriyel vardiyalarda açık işlerin ve uyarıların unutulmasını azaltmak için bir devir uygulaması geliştirdim. Kayıtlar vardiya bittiğinde kaybolmuyor; değişiklikler kişi ve tarih bilgisiyle izleniyor. AI önerilerini insan onayından ayırdım ve eşzamanlı güncellemeleri veritabanı kurallarıyla korudum."

"Neden React'e geçtin?" sorusuna

"Ortak form/dialog parçalarını ve sayfalar arası taslak durumunu daha düzenli yönetmek için geçtim. API ve DB'yi aynı tuttum; oturum, fotoğraf ve dil/tema davranışlarını regresyon testleriyle korudum. Bundle artışını da ölçüp belgeledim; otomatik hızlanma iddiasında bulunmuyorum."

"AI ne kadar doğru?" sorusuna

"Henüz gerçek saha doğruluğu ölçmedim. Yapısal yanıtı ve metindeki alıntıları doğruluyorum; bu, anlamsal doğruluğun kanıtı değil. Bir sonraki adımım uzman etiketli örneklerde sınıf bazında hataları ölçmek."

Öğrenme hedefi

Önce HTTP + async/await + SQL ilişkileri; sonra transaction/sürüm kontrolü ve güvenlik; ardından test/CI ve AI değerlendirmesi. Kullanılan araç adlarından çok, çözdüğü problemi açıklayabilmen önemli.

Bu plan yeni rol sistemi, çoklu tesis veya mikroservis dönüşümünü şart koşmuyor. Babanın ve ekibinin gerçek kullanım sorunu, projenin somut hikâyesi olmaya devam edebilir.

Sayıların kaynağı ve kısa sözlük

Rehberin ölçüm tarihi 25 Ekim 2026; kod görüntüsü 5bd019c. Kaynak bağlantıları bu commit'e sabitlenmiştir.

Bilgi türü	Nasıl elde edildi?	Ne anlama gelmez?
Kod envanteri	İzlenen src/frontend dosyaları; model/migration ve route sayısı.	Kullanıcı sayısı veya kalite puanı değil.
134 geçen yerel test	25 Ekim: 47 + 21 + 66. Önceki güvenlik turu ve temiz varlık üretim kontrolü.	Kapsam yüzdesi veya tüm açıkların yokluğu değil.
Fotoğraf deneyi	Güncel derlenmiş preparePhoto + Chromium + üç sentetik dosya; bu revizyonda tekrar ölçüldü.	Saha ortalaması, kalite veya ağ hızlanması değil.
JS boyutu / audit	Dosya baytları ve hesaplanan gzip; npm audit önce/sonra JSON.	Tarayıcı hız testi veya bütün uygulamanın güvenlik sertifikası değil.
Polling / alternatifler	İstek sayısından aritmetik model; küçük ekip için gerekçeli artı/eksi.	Yük testi veya ölçülmüş teknoloji sıralaması değil.

API / endpoint

Sunucudaki belirli işlemin HTTP yöntemi + adresi. POST /login ile GET /shifts aynı şey değildir.

Middleware / ORM

Middleware handler'dan önce/sonra çalışan kontrol katmanı; ORM uygulamadaki veri yapılarıyla SQL işlemleri arasında araçtır.

Transaction / migration

Transaction bir grup işlemi birlikte tamamlar/geri alır. Migration veritabanı yapısının sürümlü değişikliğidir.

Idempotency / optimistic lock

Idempotency aynı niyetin tekrarında ikinci kayıt üretmemek; optimistic lock eski sürümle farklı veri ezilmesini reddetmektir.

Gerekçeler bugünkü kodu ve konuşmada belirlenen hedefleri açıklar. Node/Express/Prisma ilk iskelette zaten bulunan tercihlerdi; tüm başlangıç teknolojilerinin bu rehber hazırlanırken seçildiği iddia edilmiyor. Bu rehber bir güvenlik sertifikası değildir.